

算法模板

目录

1. 并查集
2. 树状数组
3. 线段树
4. 单调队列优化 DP
5. LCA 与树链剖分
6. 最短路
7. 最小生成树
8. Tarjan
9. 二分图匹配
10. 网络流
11. 字符串
12. 数学
13. 组合数与模运算

并查集

普通并查集

```
struct DSU {  
    vector<int> p, sz; // p: 父节点, sz: 连通块大小  
  
    DSU(int n = 0) { init(n); }  
  
    void init(int n) {  
        p.resize(n + 1);  
        sz.assign(n + 1, 1);  
        iota(p.begin(), p.end(), 0); // 初始每个点都是自己的根  
    }  
  
    int find(int x) {  
        return p[x] == x ? x : p[x] = find(p[x]); // 路径压缩  
    }  
  
    bool unite(int a, int b) {  
        a = find(a), b = find(b);  
        if (a == b) return false;  
        if (sz[a] < sz[b]) swap(a, b); // 小树合并到大树  
        p[b] = a;  
        sz[a] += sz[b];  
        return true;  
    }  
  
    bool same(int a, int b) {  
        return find(a) == find(b);  
    }  
};
```

```
    }
};
```

带权并查集

$d[x]$ 表示 x 到父节点的权值，路径压缩后表示 x 到根的权值。适合维护距离、模关系、奇偶关系。

```
struct WeightedDSU {
    vector<int> p, sz;
    vector<long long> d; // d[x]: x 到父节点的权值

    WeightedDSU(int n = 0) { init(n); }

    void init(int n) {
        p.resize(n + 1);
        sz.assign(n + 1, 1);
        d.assign(n + 1, 0);
        iota(p.begin(), p.end(), 0);
    }

    int find(int x) {
        if (p[x] == x) return x;
        int fa = p[x];
        int root = find(fa);
        d[x] += d[fa]; // 压缩时累加到根的权值
        return p[x] = root;
    }

    // 添加约束: weight[b] - weight[a] = w
    bool unite(int a, int b, long long w) {
        int ra = find(a), rb = find(b);
        long long da = d[a], db = d[b]; // 此时 d[a], d[b] 已是到根的权值
        if (ra == rb) return db - da == w; // 同集合时检查是否矛盾

        if (sz[ra] < sz[rb]) {
            p[ra] = rb;
            d[ra] = db - da - w;
            sz[rb] += sz[ra];
        } else {
            p[rb] = ra;
            d[rb] = da + w - db;
            sz[ra] += sz[rb];
        }
        return true;
    }
};
```

树状数组

单点加，前缀和

```

template <class T>
struct Fenwick {
    int n;
    vector<T> tr; // tr[x] 维护一段后缀长度为 lowbit(x) 的和

    Fenwick(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        tr.assign(n + 1, T());
    }

    void add(int x, T v) {
        for (; x <= n; x += x & -x) tr[x] += v; // 向上更新所有包含 x 的区间
    }

    T sum(int x) const {
        T res = T();
        for (; x > 0; x -= x & -x) res += tr[x]; // 拆分前缀 [1, x]
        return res;
    }

    T range_sum(int l, int r) const {
        return sum(r) - sum(l - 1);
    }
};

```

区间加，区间和

维护差分数组 $b[i]$ ，原数组前缀和为 $sum(i) = (i + 1) * sum(b) - sum(i * b)$ 。

```

template <class T>
struct RangeFenwick {
    int n;
    Fenwick<T> t1, t2; // t1 维护 b, t2 维护 i * b[i]

    RangeFenwick(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        t1.init(n);
        t2.init(n);
    }

    void add_prefix(int x, T v) {
        if (x <= 0) return;
        t1.add(x, v);
        t2.add(x, v * x);
    }
};

```

```

void range_add(int l, int r, T v) {
    add_prefix(l, v);          // 差分 b[l] += v
    add_prefix(r + 1, -v);     // 差分 b[r + 1] -= v
}

T prefix_sum(int x) const {
    return (x + 1) * t1.sum(x) - t2.sum(x);
}

T range_sum(int l, int r) const {
    return prefix_sum(r) - prefix_sum(l - 1);
}
};

```

线段树

区间加，区间和

```

template <class T>
struct SegTree {
    struct Node {
        T sum = 0, lazy = 0; // sum: 区间和, lazy: 尚未下传的区间加标记
    };

    int n;
    vector<Node> tr;

    SegTree(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        tr.assign(n * 4 + 4, Node());
    }

    void build(const vector<T>& a, int u, int l, int r) {
        if (l == r) {
            tr[u].sum = a[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(a, u << 1, l, mid);
        build(a, u << 1 | 1, mid + 1, r);
        pull(u);
    }

    void pull(int u) {
        tr[u].sum = tr[u << 1].sum + tr[u << 1 | 1].sum;
    }

    void apply(int u, int l, int r, T v) {
        tr[u].sum += v * (r - l + 1);
    }
};

```

```

        tr[u].lazy += v; // 整段加, 先打懒标记
    }

    void push(int u, int l, int r) {
        if (tr[u].lazy == 0 || l == r) return;
        int mid = (l + r) >> 1;
        apply(u << 1, l, mid, tr[u].lazy);
        apply(u << 1 | 1, mid + 1, r, tr[u].lazy);
        tr[u].lazy = 0;
    }

    void range_add(int u, int l, int r, int ql, int qr, T v) {
        if (ql <= l && r <= qr) {
            apply(u, l, r, v);
            return;
        }
        push(u, l, r); // 访问子节点前先下传
        int mid = (l + r) >> 1;
        if (ql <= mid) range_add(u << 1, l, mid, ql, qr, v);
        if (qr > mid) range_add(u << 1 | 1, mid + 1, r, ql, qr, v);
        pull(u);
    }

    T query(int u, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tr[u].sum;
        push(u, l, r);
        int mid = (l + r) >> 1;
        T res = 0;
        if (ql <= mid) res += query(u << 1, l, mid, ql, qr);
        if (qr > mid) res += query(u << 1 | 1, mid + 1, r, ql, qr);
        return res;
    }
};

```

扫描线求矩形面积并

```

struct Scanline {
    struct Event {
        double x, y1, y2;
        int tag; // +1 表示左边界加入, -1 表示右边界删除
        bool operator<(const Event& other) const {
            return x < other.x;
        }
    };
};

struct Node {
    int cover = 0; // 当前区间被覆盖次数
    double len = 0; // 当前区间实际被覆盖的 y 长度
};

vector<double> ys;

```

```

vector<Node> tr;
vector<Event> events;

void add_rect(double x1, double y1, double x2, double y2) {
    events.push_back({x1, y1, y2, 1});
    events.push_back({x2, y1, y2, -1});
    ys.push_back(y1);
    ys.push_back(y2);
}

void pull(int u, int l, int r) {
    if (tr[u].cover) tr[u].len = ys[r] - ys[l - 1]; // 被至少一个矩形完全覆盖
    else if (l == r) tr[u].len = 0;
    else tr[u].len = tr[u << 1].len + tr[u << 1 | 1].len;
}

void update(int u, int l, int r, int ql, int qr, int v) {
    if (ql <= l && r <= qr) {
        tr[u].cover += v;
        pull(u, l, r);
        return;
    }
    int mid = (l + r) >> 1;
    if (ql <= mid) update(u << 1, l, mid, ql, qr, v);
    if (qr > mid) update(u << 1 | 1, mid + 1, r, ql, qr, v);
    pull(u, l, r);
}

double solve() {
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
    sort(events.begin(), events.end());
    int m = (int)ys.size() - 1; // m 个离散化后的 y 段
    tr.assign(m * 4 + 4, Node());

    double ans = 0;
    for (int i = 0; i < (int)events.size(); i++) {
        if (i) ans += tr[1].len * (events[i].x - events[i - 1].x);
        int l = lower_bound(ys.begin(), ys.end(), events[i].y1) - ys.begin() +
1;

        int r = lower_bound(ys.begin(), ys.end(), events[i].y2) - ys.begin();
        if (l <= r) update(1, 1, m, l, r, events[i].tag);
    }
    return ans;
}
};

```

单调队列优化 DP

形如: $f[i] = \max/\min(g[j] + val(i))$, 其中 j 在滑动窗口内, 且 $val(i)$ 与 j 无关。

```
// 最大子段和长度不超过 m:  $\max(s[i] - \min s[j]), i - m \leq j < i$ 
long long max_subarray_sum_with_limit(const vector<long long>& a, int m) {
    int n = (int)a.size() - 1; // a 使用 1 下标
    vector<long long> s(n + 1);
    for (int i = 1; i <= n; i++) s[i] = s[i - 1] + a[i];

    deque<int> q; // 存前缀和下标, 保证 s[q] 单调递增
    q.push_back(0);
    long long ans = LLONG_MIN;
    for (int i = 1; i <= n; i++) {
        while (!q.empty() && q.front() < i - m) q.pop_front(); // 弹出过期下标
        ans = max(ans, s[i] - s[q.front()]);
        while (!q.empty() && s[q.back()] >= s[i]) q.pop_back(); // 维护最小前缀和
        q.push_back(i);
    }
    return ans;
}
```

LCA 与树链剖分

倍增 LCA

```
struct LCA {
    int n, lg;
    vector<int> depth;
    vector<vector<int>> up, g; // up[k][u]: u 的  $2^k$  级祖先

    LCA(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        lg = 1;
        while ((1 << lg) <= n) lg++;
        g.assign(n + 1, {});
        depth.assign(n + 1, 0);
        up.assign(lg, vector<int>(n + 1, 0));
    }

    void add_edge(int a, int b) {
        g[a].push_back(b);
        g[b].push_back(a);
    }

    void dfs(int u, int fa) {
        up[0][u] = fa;
        depth[u] = depth[fa] + 1;
        for (int k = 1; k < lg; k++) up[k][u] = up[k - 1][up[k - 1][u]];
        for (int v : g[u]) {
            if (v == fa) continue;
            dfs(v, u);
        }
    }
}
```

```

    }
}

int lca(int a, int b) const {
    if (depth[a] < depth[b]) swap(a, b);
    int diff = depth[a] - depth[b];
    for (int k = 0; k < lg; k++)
        if (diff >> k & 1) a = up[k][a]; // 先跳到同一深度
    if (a == b) return a;

    for (int k = lg - 1; k >= 0; k--) {
        if (up[k][a] != up[k][b]) {
            a = up[k][a];
            b = up[k][b];
        }
    }
    return up[0][a];
}
};

```

树链剖分编号

配合线段树维护路径/子树信息。 $\text{dfn}[u]$ 到 $\text{dfn}[u] + \text{sz}[u] - 1$ 是 u 的子树。

```

struct HLD {
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> parent, depth, heavy, sz, top, dfn, rev;

    HLD(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        parent.assign(n + 1, 0);
        depth.assign(n + 1, 0);
        heavy.assign(n + 1, 0); // heavy[u]: u 的重儿子
        sz.assign(n + 1, 0);
        top.assign(n + 1, 0); // top[u]: u 所在重链的链顶
        dfn.assign(n + 1, 0); // 树上点映射到线段树位置
        rev.assign(n + 1, 0);
        timer = 0;
    }

    void add_edge(int a, int b) {
        g[a].push_back(b);
        g[b].push_back(a);
    }

    void dfs1(int u, int fa) {
        parent[u] = fa;
    }
}

```

```

    depth[u] = depth[fa] + 1;
    sz[u] = 1;
    for (int v : g[u]) {
        if (v == fa) continue;
        dfs1(v, u);
        sz[u] += sz[v];
        if (!heavy[u] || sz[v] > sz[heavy[u]]) heavy[u] = v;
    }
}

void dfs2(int u, int chain_top) {
    top[u] = chain_top;
    dfn[u] = ++timer;
    rev[timer] = u;
    if (heavy[u]) dfs2(heavy[u], chain_top); // 重儿子继承链顶
    for (int v : g[u]) {
        if (v == parent[u] || v == heavy[u]) continue;
        dfs2(v, v); // 轻儿子新开一条链
    }
}

template <class F>
void path_apply(int a, int b, F apply) {
    while (top[a] != top[b]) {
        if (depth[top[a]] < depth[top[b]]) swap(a, b);
        apply(dfn[top[a]], dfn[a]);
        a = parent[top[a]];
    }
    if (depth[a] > depth[b]) swap(a, b);
    apply(dfn[a], dfn[b]);
}

};

```

最短路

Dijkstra

```

using PII = pair<long long, int>;
const long long INF = (1LL << 62);

vector<long long> dijkstra(int n, const vector<vector<pair<int, int>>>& g, int s)
{
    vector<long long> dist(n + 1, INF);
    vector<char> done(n + 1, false); // done[u] 表示 u 的最短路已确定
    priority_queue<PII, vector<PII>, greater<PII>> pq;

    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
    }
}

```

```

        if (done[u]) continue;
        done[u] = true;
        for (auto [v, w] : g[u]) {
            if (dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

```

SPFA 判负环

```

bool has_negative_cycle(int n, const vector<vector<pair<int, int>>>& g) {
    vector<long long> dist(n + 1, 0);
    vector<int> cnt(n + 1, 0); // cnt[v]: 当前最短路经过的边数
    vector<char> inq(n + 1, true);
    queue<int> q;

    for (int i = 1; i <= n; i++) q.push(i); // 超级源点写法: 所有点入队
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                cnt[v] = cnt[u] + 1;
                if (cnt[v] >= n) return true; // 超过 n - 1 条边, 存在负环
                if (!inq[v]) {
                    q.push(v);
                    inq[v] = true;
                }
            }
        }
    }
    return false;
}

```

差分约束

把限制转成边:

- $x_b \leq x_a + c$: 连边 $a \rightarrow b, c$
- 求最大解: 建 $dist[b] \leq dist[a] + c$, 跑最短路
- 求最小解: 转成 $x_a \geq x_b - c$, 跑最长路
- 有负环说明无解

最小生成树

Kruskal

```

struct Edge {
    int a, b;
    long long w;
    bool operator<(const Edge& other) const {
        return w < other.w;
    }
};

pair<long long, vector<Edge>> kruskal(int n, vector<Edge> edges) {
    sort(edges.begin(), edges.end()); // 按边权从小到大选
    DSU dsu(n);
    long long cost = 0;
    vector<Edge> tree;
    for (auto e : edges) {
        if (dsu.unite(e.a, e.b)) { // 不成环才加入
            cost += e.w;
            tree.push_back(e);
        }
    }
    if ((int)tree.size() != n - 1) return {-1, {}}; // 图不连通
    return {cost, tree};
}

```

Tarjan

强连通分量 SCC

```

struct SCC {
    int n, timer = 0, scc_cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, stk, id, sz;
    vector<char> in_stk;

    SCC(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        id.assign(n + 1, 0); // id[u]: u 所属 SCC 编号
        sz.assign(n + 1, 0); // sz[c]: 第 c 个 SCC 的大小
        in_stk.assign(n + 1, false);
        stk.clear();
        timer = scc_cnt = 0;
    }

    void add_edge(int a, int b) {

```

```

        g[a].push_back(b);
    }

    void dfs(int u) {
        dfn[u] = low[u] = ++timer;
        stk.push_back(u);
        in_stk[u] = true;
        for (int v : g[u]) {
            if (!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_stk[v]) {
                low[u] = min(low[u], dfn[v]); // 只能用栈内点更新
            }
        }
        if (dfn[u] == low[u]) { // u 是一个 SCC 的最高点
            scc_cnt++;
            while (true) {
                int x = stk.back();
                stk.pop_back();
                in_stk[x] = false;
                id[x] = scc_cnt;
                sz[scc_cnt]++;
                if (x == u) break;
            }
        }
    }

    void solve() {
        for (int i = 1; i <= n; i++)
            if (!dfn[i]) dfs(i);
    }
};

```

割点

```

struct CutVertex {
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> dfn, low;
    vector<char> is_cut;

    CutVertex(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        is_cut.assign(n + 1, false);
        timer = 0;
    }
};

```

```

    }

    void add_edge(int a, int b) {
        g[a].push_back(b);
        g[b].push_back(a);
    }

    void dfs(int u, int root) {
        dfn[u] = low[u] = ++timer;
        int child = 0; // root 需要单独看子树数量
        for (int v : g[u]) {
            if (!dfn[v]) {
                child++;
                dfs(v, root);
                low[u] = min(low[u], low[v]);
                if (u != root && low[v] >= dfn[u]) is_cut[u] = true;
            } else {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (u == root && child > 1) is_cut[u] = true;
    }

    void solve() {
        for (int i = 1; i <= n; i++)
            if (!dfn[i]) dfs(i, i);
    }
};

```

二分图匹配

匈牙利算法

左部点 $1..n1$, 右部点 $1..n2$ 。

```

struct Hungarian {
    int n1, n2;
    vector<vector<int>> g;
    vector<int> match; // match[v]: 右部点 v 当前匹配的左部点
    vector<char> vis;

    Hungarian(int n1 = 0, int n2 = 0) { init(n1, n2); }

    void init(int n1_, int n2_) {
        n1 = n1_, n2 = n2_;
        g.assign(n1 + 1, {});
        match.assign(n2 + 1, 0);
    }

    void add_edge(int a, int b) {
        g[a].push_back(b);
    }
};

```

```

    }

    bool dfs(int u) {
        for (int v : g[u]) {
            if (vis[v]) continue;
            vis[v] = true;
            if (!match[v] || dfs(match[v])) { // 找到空点或让原匹配点改配
                match[v] = u;
                return true;
            }
        }
        return false;
    }

    int solve() {
        int ans = 0;
        for (int i = 1; i <= n1; i++) {
            vis.assign(n2 + 1, false);
            if (dfs(i)) ans++;
        }
        return ans;
    }
};

```

网络流

Dinic 最大流

```

struct Dinic {
    struct Edge {
        int to, rev;    // rev 是反向边在 g[to] 中的位置
        long long cap;  // 当前残量
    };

    int n, s, t;
    vector<vector<Edge>> g;
    vector<int> level, it; // level: 分层图, it: 当前弧优化

    Dinic(int n = 0) { init(n); }

    void init(int n_) {
        n = n_;
        g.assign(n + 1, {});
        level.assign(n + 1, 0);
        it.assign(n + 1, 0);
    }

    void add_edge(int u, int v, long long c) {
        Edge a{v, (int)g[v].size(), c};
        Edge b{u, (int)g[u].size(), 0};
        g[u].push_back(a);
    }
};

```

```

        g[v].push_back(b);
    }

    bool bfs() {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto &e : g[u]) {
                if (e.cap > 0 && level[e.to] == -1) {
                    level[e.to] = level[u] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    }

    long long dfs(int u, long long f) {
        if (u == t) return f;
        for (int &i = it[u]; i < (int)g[u].size(); i++) {
            Edge &e = g[u][i];
            if (e.cap <= 0 || level[e.to] != level[u] + 1) continue;
            long long ret = dfs(e.to, min(f, e.cap));
            if (ret) {
                e.cap -= ret;
                g[e.to][e.rev].cap += ret; // 增加反向边残量, 方便撤流
                return ret;
            }
        }
        return 0;
    }

    long long maxflow(int s_, int t_) {
        s = s_, t = t_;
        long long flow = 0;
        while (bfs()) {
            fill(it.begin(), it.end(), 0);
            while (long long f = dfs(s, LLONG_MAX)) flow += f;
        }
        return flow;
    }
};

```

字符串

KMP

```

vector<int> prefix_function(const string& p) {
    int n = (int)p.size();
    vector<int> ne(n); // ne[i]: p[0..i] 的最长相等真前后缀长度
    for (int i = 1, j = 0; i < n; i++) {
        while (j && p[i] != p[j]) j = ne[j - 1];
        if (p[i] == p[j]) j++;
        ne[i] = j;
    }
    return ne;
}

vector<int> kmp_match(const string& s, const string& p) {
    vector<int> ne = prefix_function(p), pos;
    for (int i = 0, j = 0; i < (int)s.size(); i++) {
        while (j && s[i] != p[j]) j = ne[j - 1];
        if (s[i] == p[j]) j++;
        if (j == (int)p.size()) {
            pos.push_back(i - j + 1); // 匹配起点, 0 下标
            j = ne[j - 1];
        }
    }
    return pos;
}

```

字符串哈希

```

struct StringHash {
    using ull = unsigned long long;
    static const ull base = 131;
    vector<ull> h, p; // h: 前缀哈希, p: base 的幂

    StringHash(const string& s = "") { init(s); }

    void init(const string& s) {
        int n = (int)s.size();
        h.assign(n + 1, 0);
        p.assign(n + 1, 1);
        for (int i = 1; i <= n; i++) {
            h[i] = h[i - 1] * base + (unsigned char)s[i - 1];
            p[i] = p[i - 1] * base;
        }
    }

    ull get(int l, int r) const {
        // 查询 [l, r] 下标闭区间的哈希
        return h[r] - h[l - 1] * p[r - l + 1];
    }
};

```

AC 自动机

```

struct AhoCorasick {
    struct Node {
        array<int, 26> next{}; // next[c]: 字符 c 的转移
        int fail = 0, cnt = 0; // fail: 失配指针, cnt: 以该点结尾的模式串个数
    };

    vector<Node> tr;

    AhoCorasick() {
        tr.push_back(Node()); // 0 号点是根
    }

    void insert(const string& s) {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!tr[u].next[c]) {
                tr[u].next[c] = (int)tr.size();
                tr.push_back(Node());
            }
            u = tr[u].next[c];
        }
        tr[u].cnt++;
    }

    void build() {
        queue<int> q;
        for (int c = 0; c < 26; c++) {
            int v = tr[0].next[c];
            if (v) q.push(v);
        }
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int c = 0; c < 26; c++) {
                int v = tr[u].next[c];
                if (v) {
                    tr[v].fail = tr[tr[u].fail].next[c];
                    q.push(v);
                } else {
                    tr[u].next[c] = tr[tr[u].fail].next[c]; // 补全自动机转移
                }
            }
        }
    }

    long long query(const string& s) {
        long long ans = 0;
        int u = 0;
        for (char ch : s) {

```

```

        u = tr[u].next[ch - 'a'];
        for (int j = u; j && tr[j].cnt != -1; j = tr[j].fail) {
            ans += tr[j].cnt;
            tr[j].cnt = -1; // 防止同一个模式串重复统计
        }
    }
    return ans;
}
};

```

数学

线性筛

```

struct LinearSieve {
    vector<int> primes, minp; // minp[x]: x 的最小质因子

    LinearSieve(int n = 0) {
        if (n) init(n);
    }

    void init(int n) {
        primes.clear();
        minp.assign(n + 1, 0);
        for (int i = 2; i <= n; i++) {
            if (!minp[i]) {
                minp[i] = i;
                primes.push_back(i);
            }
            for (int p : primes) {
                if (p > minp[i] || 1LL * i * p > n) break;
                minp[i * p] = p; // 每个合数只被最小质因子筛掉一次
            }
        }
    }
};

```

扩展欧几里得与逆元

```

long long exgcd(long long a, long long b, long long& x, long long& y) {
    if (!b) {
        x = 1, y = 0;
        return a;
    }
    long long x1, y1;
    long long g = exgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - a / b * y1;
    return g; // ax + by = gcd(a, b)
}

```

```

}

long long inverse(long long a, long long mod) {
    long long x, y;
    long long g = exgcd(a, mod, x, y);
    if (g != 1) return -1; // 不互质则逆元不存在
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

```

中国剩余定理，非互质版

合并两个方程： $x = a_1 \pmod{m_1}$, $x = a_2 \pmod{m_2}$ 。

```

bool merge_crt(long long& a1, long long& m1, long long a2, long long m2) {
    long long x, y;
    long long g = exgcd(m1, m2, x, y);
    long long c = a2 - a1;
    if (c % g) return false; // 差值不能被 gcd 整除，无解

    long long mod = m2 / g;
    x = (__int128)x * (c / g) % mod;
    a1 += x * m1;
    m1 *= mod; // 新模数为 lcm(m1, m2)
    a1 = (a1 % m1 + m1) % m1;
    return true;
}

// equations: {余数, 模数}
pair<long long, long long> exCRT(const vector<pair<long long, long long>>&
equations) {
    long long a = equations[0].first;
    long long m = equations[0].second;
    for (int i = 1; i < (int)equations.size(); i++) {
        if (!merge_crt(a, m, equations[i].first, equations[i].second)) return {-1,
-1};
    }
    return {a, m}; // x = a (mod m)
}

```

矩阵快速幂

```

using Matrix = vector<vector<long long>>;

Matrix multiply(const Matrix& a, const Matrix& b, long long mod) {
    int n = (int)a.size();
    int m = (int)b[0].size();
    int k = (int)b.size();

```

```

Matrix c(n, vector<long long>(m, 0));
for (int i = 0; i < n; i++) {
    for (int t = 0; t < k; t++) {
        if (!a[i][t]) continue;
        for (int j = 0; j < m; j++) {
            c[i][j] = (c[i][j] + (__int128)a[i][t] * b[t][j]) % mod;
        }
    }
}
return c;
}

Matrix matrix_power(Matrix a, long long b, long long mod) {
    int n = (int)a.size();
    Matrix res(n, vector<long long>(n, 0));
    for (int i = 0; i < n; i++) res[i][i] = 1; // 单位矩阵
    while (b) {
        if (b & 1) res = multiply(res, a, mod);
        a = multiply(a, a, mod);
        b >>= 1;
    }
    return res;
}

```

高斯消元

返回 0 无解, 1 唯一解, 2 无穷多解。

```

int gauss(vector<vector<double>>& a, vector<double>& ans) {
    const double eps = 1e-9;
    int n = (int)a.size();
    int m = (int)a[0].size() - 1; // 最后一列是常数项
    int row = 0, col = 0;
    vector<int> where(m, -1); // where[col]: 主元所在行

    while (row < n && col < m) {
        int sel = row;
        for (int i = row; i < n; i++)
            if (fabs(a[i][col]) > fabs(a[sel][col])) sel = i;
        if (fabs(a[sel][col]) < eps) {
            col++;
            continue;
        }
        swap(a[sel], a[row]);
        where[col] = row;

        double div = a[row][col];
        for (int j = col; j <= m; j++) a[row][j] /= div; // 主元化为 1
        for (int i = 0; i < n; i++) {
            if (i == row) continue;
            double factor = a[i][col];

```

```

        for (int j = col; j <= m; j++) a[i][j] -= factor * a[row][j];
    }
    row++, col++;
}

ans.assign(m, 0);
for (int i = 0; i < m; i++)
    if (where[i] != -1) ans[i] = a[where[i]][m];

for (int i = 0; i < n; i++) {
    double sum = 0;
    for (int j = 0; j < m; j++) sum += ans[j] * a[i][j];
    if (fabs(sum - a[i][m]) > eps) return 0;
}
return row < m ? 2 : 1;
}

```

组合数与模运算

快速幂

```

long long qpow(long long a, long long b, long long mod) {
    long long res = 1 % mod;
    a %= mod;
    while (b) {
        if (b & 1) res = (__int128)res * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1;
    }
    return res;
}

```

组合数预处理

模数需要是质数。

```

struct Comb {
    long long mod;
    vector<long long> fac, ifac; // fac: 阶乘, ifac: 阶乘逆元

    Comb(int n = 0, long long mod_ = 1000000007LL) {
        init(n, mod_);
    }

    void init(int n, long long mod_) {
        mod = mod_;
        fac.assign(n + 1, 1);
        ifac.assign(n + 1, 1);
        for (int i = 1; i <= n; i++) fac[i] = fac[i - 1] * i % mod;
    }
}

```

```

        ifac[n] = qpow(fac[n], mod - 2, mod);
        for (int i = n; i >= 1; i--) ifac[i - 1] = ifac[i] * i % mod;
    }

    long long C(int n, int k) const {
        if (k < 0 || k > n) return 0;
        return fac[n] * ifac[k] % mod * ifac[n - k] % mod;
    }
};

```

Lucas

```

long long lucas(long long n, long long k, long long p) {
    Comb comb((int)p - 1, p);
    function<long long(long long, long long)> dfs = [&](long long a, long long b)
-> long long {
        if (!b) return 1;
        return comb.C(a % p, b % p) * dfs(a / p, b / p) % p;
    };
    return dfs(n, k);
}

```

Catalan 数

常见形式：合法括号序列、出栈序列、非交叉连线。

```

long long catalan(int n, const Comb& comb) {
    return (comb.C(2 * n, n) - comb.C(2 * n, n - 1) + comb.mod) % comb.mod;
}

```